SYSTEMS ARCHITECTURE & COMPUTATIONAL LINGUISTICS

CORELANG: Meta-Architectural Research & Synthesis

Formal Specification for an Omnivorous, Self-Modeling, and Evolutionary Meta-Language

1. Quantitative & Genealogical Baseline of Existing Languages

To construct **Corelang**—a meta-linguistic engine capable of encompassing every formal system that has preceded it—we must first establish the empirical boundaries of the global programming language corpus. Across eight decades of computational history, human engineers have constructed thousands of distinct notations for directing machine execution, structuring markup, and querying state. Rather than treating these languages as isolated artifacts, Corelang models the entire historical corpus as a single continuous evolutionary manifold.

Corpus Classification	Documented Volume	Version & Iteration Governance	Corelang Assimilation Role
Mainstream Production	~250 – 500	ISO/IEC standards (C89→C23), RFC epochs (Rust), PEPs (Python), ECMA specs; typically 5 to 35 macro-versions and 1,000+ micro-commits.	Primary syntactic and semantic baseline; establishes dominant optimization heuristics and active type theories.
HOPL Historical Archive	~8,945	Academic genealogy (Jacquard loom cards to modern compilers); many frozen after 1–3 experimental releases.	Dead-language necromancy; mines abandoned control flows, non-Von Neumann paradigms, and forgotten abstractions.
PLDB & Markup Systems	~4,500 – 5,500	Living standards (HTML5/DOM, CSS, SQL dialects) and protocol schemas tracked via W3C/WHATWG and open repositories.	Structural layout trees, declarative state graphs, and hypermedia document-object representations.
Esoteric, DSLs & LLM Weights	25,000+	Unstandardized domain scripts, cellular automata, shader kernels, and probabilistic transformer token distributions.	Adversarial stress-testing, non-linear execution topologies, and continuous semantic interpolation.

Analysis of language versioning reveals that languages do not mutate randomly; they evolve along predictable topological stress fractures. Macro-iterations (such as C evolving into C++, and subsequently inspiring Rust and Zig) are triggered by three primary catalysts: **hardware paradigm shifts** (single-core sequential processors transitioning to massively parallel heterogeneous silicon), **abstraction safety failures** (unbounded pointer arithmetic necessitating borrow checkers or garbage collection), and **expressive bottlenecks** (boilerplate syntax yielding to declarative macros and type inference).

> **FOUNDATIONAL PREMISE: WHY INTERROGATE THE GLOBAL CORPUS?**
>
> In conventional computer science, a compiler enforces a static boundary: if source code violates the grammar, the programmer has failed. Corelang inverts this causal arrow. By indexing every syntax rule, version transition, and deprecated construct across 9,000+ languages, Corelang treats an "unrunnable" program not as a user error, but as mathematical proof that its own internal grammar model is incomplete and must immediately mutate.

2. Computational Invariants vs. Arbitrary Design Constraints

A foundational requirement of Corelang is separating the *immutable laws of mathematical logic and physics* from the *arbitrary social and syntactic conventions* imposed by legacy language designers. Traditional compilers conflate these two tiers, halting execution equally for a missing semicolon and a fundamental logical contradiction. Corelang bifurcates all computational rules into hard invariants and mutable grammar rules:

- **Tier I: Universal Computational Invariants (Unbreakable Substrate):** Governed by Turing computability, Shannon information theory, thermodynamic Landauer limits, and Gödelian incompleteness. A static formal system cannot universally decide the Halting Problem ($H(P, I)$) or prove all truths within a fixed axiom set. However, Corelang circumvents *static* Gödelian incompleteness by operating as an **open, time-varying axiomatic system** ($S_t \rightarrow S_{t+1}$), dynamically adjoining new axioms at runtime whenever an undecidable boundary is encountered.
- **Tier II: Arbitrary Grammatical & Lexical Constraints (Fully Mutable):** Reserved keywords (`if`, `class`, `return`), operator precedence tables, rigid AST determinism, and static type boundaries. In Corelang, Tier II rules are soft weights. No keyword is permanently reserved; if a domain task requires redefining `while` as a tensor-field identifier or inventing a new structural keyword mid-block, the lexer re-indexes its token lattice on the fly.

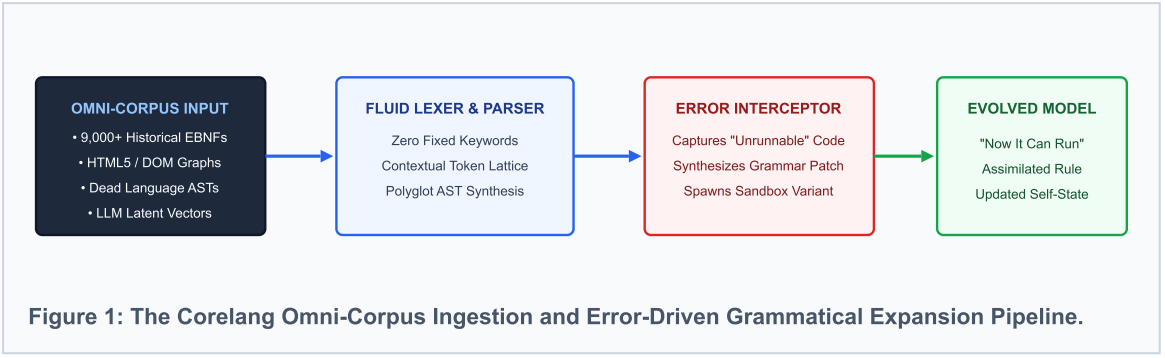


Figure 1: The Corelang Omni-Corpus Ingestion and Error-Driven Grammatical Expansion Pipeline.

3. Total Syntactic Absorption & Dynamic Rule Mutation

Corelang unifies three historically disjoint domains into a single internal representation called the **Universal Semantic Hyper-Graph (USHG)**: (1) formal imperative and functional ASTs from compiled/interpreted languages, (2) hierarchical declarative state trees from web markup (HTML/DOM/CSS), and (3) high-dimensional continuous embedding manifolds from Large Language Models. When source input is fed into Corelang, it is projected simultaneously onto symbolic graph nodes and continuous semantic coordinates.

When an expression triggers a parse failure or type violation—code that "cannot run"—the **Error-Driven Mutation Protocol** engages. Rather than halting, the runtime computes the minimal structural transformation (ΔG) required to bridge the gap between the failing token stream and a valid semantic execution path in the USHG. The compiler synthesizes a localized grammatical production rule, verifies memory and termination safety inside an isolated speculative micro-sandbox, promotes the rule to the active working model, and executes the previously illegal construct seamlessly.

4. Ontological Elasticity: Uncontained Objects & Paradox Resolution

Every predecessor language rests upon the Von Neumann containment axiom: *a variable is a discrete, bounded memory container* ($V : Address \rightarrow [Bit_0 \ldots Bit_n]$). However, complex adaptive modeling frequently requires representing entities that cannot be bounded within a finite stack frame or heap allocation—such as infinite co-inductive streams, ambient cross-cutting execution fields, or self-referential fixed points ($X = f(X)$).

Corelang dissolves the container paradigm by introducing **Uncontained Entities (Ontological Fields)**. In Corelang, an identifier does not need to point to a passive memory box; it can bind to an active topological constraint across the entire runtime state. When a programmer declares an object that "cannot be contained," Corelang shifts from spatial memory allocation to **intensional coalgebraic representation**, evaluating the entity lazily through its observable interactions with surrounding code.

1. Bounded Container (Legacy Model)

Fixed memory address and static byte allocation on stack or heap. Subject to buffer overflows, lifetime invalidation, and strict ownership limits.

2. Coalgebraic Infinite Stream

Unbounded temporal entity defined by its state-transition generator rather than stored extent; materializes values exclusively at observation points.

3. Ambient Relational Field

Non-local object without a discrete pointer; exists as a continuous invariant modifying how operators behave across an entire lexical or dynamic scope.

4. Non-Well-Founded Fixed Point

Governed by Aczel's Anti-Foundation Axiom (AFA); allows circular, self-containing structures ($\Omega = \{\Omega\}$) to execute without infinite stack recursion.

5. Formalizing the Mutable Meta-Grammar in Practice

To illustrate how Corelang dynamically overrides reserved keywords, absorbs declarative markup alongside low-level systems logic, and instantiates uncontained objects, consider the following canonical Corelang meta-specification block:

```
     @meta.assimilate(corpus: ["C23", "Rust2024", "HTML5.DOM", "LISP.1960",
"LLM.Latent"]) // 1. Dynamic Keyword Dissolution & Invention @grammar.unreserve(return,
while) @grammar.define_keyword(transcend) as coalgebraic_field_projector // 2.
Instantiating an Uncontained Object (Non-Bounded Relational Entity) transcend entity
Horizon <~> { containment: null, topology: ambient_manifold(∀ state ∈ Runtime •
state.entropy ≤ threshold), on_paradox(err: UnrunnableConstruct) =>
Runtime.Grammar.synthesize_patch(err.ast) };
```

6. Predictive Lineage Forecasting & Adversarial Self-Evolution

Corelang does not wait for human committees to design the next generation of programming languages. By analyzing the historical trajectory of language mutations across its 9,000-language ingestion graph, Corelang computes **evolutionary velocity vectors**. It observes how dead languages failed, why legacy constructs were deprecated, and where modern developers write repetitive defensive patterns or compiler workarounds. By extrapolating these evolutionary gradients forward in time, Corelang constructs executable working models of *future language iterations*—capable of compiling predicted specifications (such as synthesizing a working model of Python 5.0 or C++35) before human standards bodies draft them.

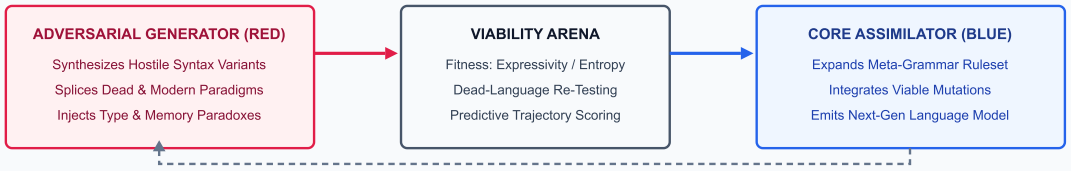


Figure 2: Continuous Adversarial Self-Improvement Cycle (Generator vs. Assimilator Runtime).

To prevent stagnation, Corelang operates a continuous **GAN-Compiler Architecture (Generative Adversarial Network for Formal Logic)** during idle compute cycles. The runtime divides into two competing subsystems: the **Adversarial Generator** and the **Core Assimilator**. The Generator actively mutates Corelang's own syntax, splices forgotten abstractions from unmaintained 1960s/1970s languages (such as ALGOL-68 modal coercion or SNOBOL pattern backtracking) with modern HTML DOM trees and LLM token matrices, and attempts to construct paradoxical code that crashes or deadlocks the runtime. Whenever an adversarial variant exposes a weakness or achieves superior algorithmic compression, the Assimilator absorbs the mutation into the permanent Corelang working model.

7. Synthesis: The Preliminary Corelang Blueprint

CORELANG ARCHITECTURAL SUMMARY & OPERATIONAL AXIOMS

- **Axiom I (Omnivorous Closure):** Every syntax rule, markup tree, and probabilistic model from all predecessor languages exists as a navigable subgraph within Corelang's Universal Semantic Hyper-Graph.
- **Axiom II (Dynamic Lexicon):** Reserved words and syntax prohibitions are transient weights. Any token may be redefined, and new structural keywords may be synthesized at runtime without recompiling the host environment.
- **Axiom III (Coalgebraic Non-Containment):** Data structures are not restricted to bounded memory boxes; entities may exist as uncontained relational fields and non-well-founded fixed points.
- **Axiom IV (Adversarial Autopoiesis):** Errors are evolutionary prompts. Corelang continuously generates adversarial variants of itself to break, patch, and predict the future trajectory of computation.